

PRAKTIKUM SISTEM OPERASI

MODUL 9
(Sistem Syscall)



Oleh:

(Ammar Dzaki Nandana)

(2311104071)

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK
DIREKTORAT KAMPUS PURWOKERTO
UNIVERSITAS TELKOM
2026

MODUL 9

1. Implementasi System Call `chname`

```
~xinu_m9/system
#include <xinu.h>

pril6 chname(pid32 pid, pril6 newprio) {
    intmask mask;
    struct procent *prptr;
    pril6 oldprio;

    mask = disable();
    if (isbadpid(pid)) {
        restore(mask);
        return (pril6) SYSERR;
    }
    prptr = &proctab[pid];
    oldprio = prptr->prprio;
    prptr->prprio = newprio;
    kprintf("\n\nNew Syscall is easy\n\n");
    restore(mask);
    return oldprio;
}
```

Pada praktikum ini dilakukan modifikasi kernel sistem operasi Xinu dengan menambahkan sebuah *system call* (*syscall*) baru bernama `chname`.

System call merupakan layanan yang disediakan oleh kernel agar program dapat meminta operasi tertentu secara langsung kepada sistem operasi. Pada percobaan ini, `syscall chname` digunakan untuk mengubah prioritas suatu proses yang sedang aktif di dalam tabel proses Xinu.

Fungsi:

```
chname(pid32 pid, pril6 newprio)
```

menerima dua parameter, yaitu:

- `pid` → identitas proses yang akan diubah.
- `newprio` → nilai prioritas baru yang akan diberikan.

Di awal fungsi dibuat beberapa variabel, yaitu:

- `mask` → menyimpan status interrupt.
- `*prptr` → pointer menuju struktur proses.
- `oldprio` → menyimpan prioritas lama sebelum diubah.

Selanjutnya sistem menjalankan:

```
disable();
```

untuk menonaktifkan interrupt sementara agar proses perubahan data kernel berlangsung aman tanpa gangguan proses lain.

Validasi PID

Setelah itu dilakukan pengecekan menggunakan:

```
isbadpid(pid)
```

untuk memastikan bahwa PID yang dimasukkan valid.

Jika PID tidak valid maka:

Interrupt dikembalikan menggunakan:

```
restore(mask);
```

Fungsi mengembalikan:

```
SYSERR
```

Sebaliknya, jika PID valid maka pointer:

```
prptr = &proctab[pid];
```

akan diarahkan ke tabel proses milik Xinu.

Prioritas lama proses disimpan ke variabel `oldprio`, kemudian prioritas baru diberikan menggunakan:

```
prptr->prprio = newprio;
```

Setelah perubahan berhasil, program menampilkan pesan:

```
New Syscall is easy
```

sebagai bukti bahwa syscall berhasil dipanggil dan dijalankan.

Mengembalikan Kondisi Sistem

Di bagian akhir fungsi, interrupt dikembalikan ke kondisi semula menggunakan:

```
restore(mask);
```

Kemudian fungsi mengembalikan nilai `oldprio`.

Pengembalian prioritas lama berguna agar pengguna dapat mengetahui nilai prioritas proses sebelum dilakukan perubahan.

Berdasarkan output terminal, setelah kernel Xinu dijalankan muncul perintah `ps` yang menampilkan daftar proses aktif seperti:

- `prnull`
- `ipout`
- `netin`

- Main process
- shell
- ps

Saat syscall dipanggil, sistem berhasil menampilkan pesan:

```
New Syscall is easy
```

Hal ini menunjukkan bahwa syscall yang dibuat berhasil ditambahkan ke kernel Xinu, berhasil dikompilasi, dan dapat dieksekusi tanpa error.

Dengan percobaan ini dapat dipahami bahwa Xinu memungkinkan pengguna melakukan modifikasi layanan kernel secara langsung, khususnya pada manajemen proses seperti perubahan prioritas proses.

2. Modifikasi System Call **chprio**

Pada praktikum ini dilakukan modifikasi terhadap file sistem:

```
xinu/system/chprio.c
```

yang berisi implementasi system call **chprio**.

Syscall ini digunakan untuk mengubah prioritas penjadwalan (*scheduling priority*) suatu proses pada sistem operasi Xinu. Prioritas proses menentukan tingkat kepentingan suatu proses ketika scheduler memilih proses yang akan dijalankan terlebih dahulu.

Semakin besar nilai prioritas, maka semakin tinggi peluang proses memperoleh jatah CPU.

Fungsi dideklarasikan sebagai:

```
syscall chprio(pid32 pid, pri16 newprio)
```

Parameter:

- `pid` → ID proses yang akan diubah prioritasnya.
- `newprio` → prioritas baru yang diberikan.

Di dalam fungsi terdapat beberapa variabel pendukung:

- `mask` → menyimpan status interrupt.
- `*prptr` → pointer menuju entri proses.
- `oldprio` → menyimpan prioritas lama.

Menonaktifkan Interrupt

Langkah pertama yang dilakukan program adalah:

```
mask = disable();
```

Perintah ini digunakan untuk menonaktifkan interrupt sementara sehingga perubahan data pada kernel dapat berlangsung aman tanpa gangguan proses lain.

Validasi Input PID

Sesuai instruksi praktikum, input `pid` harus berupa angka yang berada pada rentang 0 sampai `NPROC`.

Untuk melakukan pengecekan digunakan fungsi:

```
if (isbadpid(pid)) {  
    restore(mask);  
    return SYSERR;  
}
```

Fungsi `isbadpid(pid)` akan bernilai benar apabila:

- PID kurang dari 0.
- PID melebihi `NPROC`.
- PID tidak menunjuk ke proses yang valid.

Jika kondisi tersebut terjadi maka:

1. Interrupt dikembalikan menggunakan `restore(mask)`.
2. Syscall dibatalkan.
3. Fungsi mengembalikan `SYSERR`.

Validasi ini penting agar kernel tidak mengakses alamat process table yang tidak semestinya.

Validasi Input Prioritas Baru

Selain PID, parameter `newprio` juga harus divalidasi.

Prioritas harus berupa bilangan positif atau lebih besar dari nol.

Pada source code ditambahkan pengecekan:

```
if (newprio <= 0) {  
    restore(mask);  
    return SYSERR;  
}
```

Kondisi tersebut memastikan bahwa nilai prioritas:

- tidak boleh nol,
- tidak boleh negatif.

Jika pengguna memasukkan prioritas yang tidak valid maka syscall langsung dihentikan dan mengembalikan `SYSERR`.

Validasi ini dilakukan agar sistem penjadwalan Xinu tetap berjalan normal karena prioritas nol atau negatif tidak sesuai dengan aturan scheduling.

Proses Perubahan Prioritas

Apabila kedua input lolos validasi, maka program melanjutkan proses perubahan prioritas menggunakan kode berikut:

```
prptr = &proctab[pid];  
oldprio = prptr->prprio;  
prptr->prprio = newprio;
```

Penjelasan:

- `prptr = &proctab[pid];`
→ pointer diarahkan ke data proses sesuai PID.
- `oldprio = prptr->prprio;`
→ menyimpan prioritas lama.
- `prptr->prprio = newprio;`
→ mengganti prioritas lama dengan prioritas baru.

Dengan langkah ini, data proses di kernel berhasil diperbarui sesuai input pengguna.

Mengembalikan Kondisi Sistem

Setelah perubahan selesai dilakukan, interrupt dikembalikan menggunakan:

```
restore(mask);  
return oldprio;
```

Perintah `restore(mask)` digunakan untuk mengaktifkan kembali interrupt, sedangkan `return oldprio` digunakan untuk mengembalikan nilai prioritas lama sebagai informasi bahwa perubahan berhasil dilakukan.

3. Pengujian System Call `chprio`

Setelah syscall `chprio` berhasil dimodifikasi dan dikompilasi pada kernel Xinu, langkah berikutnya adalah melakukan pengujian menggunakan beberapa perintah shell, yaitu:

- `ps`
- `uptime`
- `ps` kembali

Pengujian dilakukan untuk membuktikan bahwa syscall `chprio` mampu mengubah prioritas proses yang dipilih, khususnya proses dengan PID = 5.

Menjalankan Perintah `ps` Sebelum Syscall Dipanggil

Langkah awal dilakukan dengan mengetik:

```
xsh $ ps
```

Perintah `ps` digunakan untuk menampilkan daftar proses aktif pada sistem Xinu beserta informasi seperti:

- PID proses
- Nama proses

- State proses
- Prioritas proses
- Parent PID
- Stack base
- Stack pointer
- Ukuran stack

Berdasarkan output awal:

- PID 5 = shell
- Prioritas = 50

Selain itu:

- ipout memiliki prioritas 500
- Main process memiliki prioritas 20
- ps memiliki prioritas 20

Nilai tersebut merupakan kondisi awal sebelum syscall dijalankan.

Menjalankan Perintah **uptime**

Langkah berikutnya adalah menjalankan:

```
xsh $ uptime
```

Perintah `uptime` digunakan untuk menampilkan lama waktu sistem aktif sejak booting.

Pada output terlihat tulisan:

```
Xinu has been up 3 second(s)
```

Namun pada praktikum ini, perintah `uptime` telah dimodifikasi sehingga memanggil syscall `chprio`.

Hal ini dibuktikan dengan munculnya pesan:

```
New Syscall is easy
```

Pesan tersebut menunjukkan bahwa syscall berhasil dijalankan.

Pada praktikum ini syscall diprogram untuk mengubah:

- prioritas proses `ipout` (PID 1) dari 500 menjadi 20,
- prioritas proses `shell` (PID 5) dari 50 menjadi 33.

Menjalankan Perintah **ps** Setelah Syscall Dipanggil

Setelah `uptime` dijalankan, pengguna kembali mengetik:

```
xsh $ ps
```

Tujuan langkah ini adalah membandingkan daftar proses setelah syscall `chprio` dieksekusi.

Pada output kedua terlihat bahwa:

- proses `shell` dengan PID 5 yang sebelumnya memiliki prioritas 50
- berubah menjadi 33

Selain itu:

- proses `ipout` yang sebelumnya memiliki prioritas 500
- berubah menjadi 20

Perubahan ini membuktikan bahwa syscall `chprio` berhasil memodifikasi nilai `prprio` pada tabel proses `proctab`.

Analisis Perubahan Prioritas PID = 5		
Sebelum syscall dijalankan:		
PID	Nama Proses	Prioritas
5	shell	50
Setelah syscall dijalankan:		
PID	Nama Proses	Prioritas
5	shell	33
Artinya, prioritas proses <code>shell</code> berhasil berubah dari <code>50</code> menjadi <code>33</code> .		

Perubahan tersebut menunjukkan bahwa:

- syscall menerima PID valid yaitu 5,
- syscall menerima prioritas baru valid yaitu 33,
- field `prprio` pada `proctab[5]` berhasil diperbarui.

Pengujian Input Salah

Setelah syscall berhasil dijalankan menggunakan input yang benar, dilakukan pengujian kembali menggunakan input yang salah untuk memastikan validasi program berjalan dengan baik.

Pengujian Prioritas Negatif

Pemanggilan:

```
chprio(5, -3)
```

Pada kondisi ini:

- PID 5 valid,

- tetapi prioritas -3 tidak valid karena prioritas harus bernilai positif.

Saat program dijalankan, syscall mendeteksi bahwa: